GIT

Sistema de Control de Versiones Distribuido

1. Control de Versiones

- **Propósito:** Registrar historial de cambios a lo largo del tiempo.
- **Seguridad:** Permite revertir archivos a estados o versiones previas.
- **Trazabilidad:** Identifica autor, fecha y motivo de cada modificación.
- **Arquitectura Distribuida:** Cada desarrollador cuenta con una copia completa del historial localmente.

2. Repositorios

- **Repositorio Local:** Carpeta de trabajo en la máquina que aloja la carpeta oculta `.git/`.
- **Repositorio Remoto:** Servidor centralizado en la nube (GitHub, GitLab, Bitbucket).
- **Flujo de sincronización:**
  → Descargar: `git clone` / `git pull`
  → Subir: `git push`

3. Ramas (Branches)

- **Concepto:** Líneas de desarrollo paralelas e independientes.
- **Rama Principal:** Rama de producción por defecto (`main` / `master`).
- **Fusión (Merge):** Unión de cambios de una rama hacia otra.
- **Conflictos:** Cuando dos ramas modifican las mismas líneas; requieren resolución manual.

4. Estados de Git

- **1. Working Directory:** Archivos modificados en tu editor sin registrar (*Modified*).
- **2. Staging Area (Index):** Archivos seleccionados listos para empaquetarse (*Staged*).
- **3. Repository (.git):** Archivos guardados definitivamente con un commit (*Committed*).

5. Ventajas de Git

- **Velocidad:** Operaciones casi instantáneas ejecutadas en local.
- **Integridad de datos:** Cada cambio se valida mediante sumas criptográficas (SHA).
- **Trabajo Offline:** No requiere internet para ver historial, ramificar ni hacer commits.
- **Colaboración fluida:** Varios programadores trabajando a la vez sin pisarse.

6. Comandos Clave

- `git init` : Inicializa repositorio local.
- `git status` : Muestra el estado del árbol de trabajo.
- `git add .` : Pasa cambios al Staging Area.
- `git commit -m "..."` : Guarda cambios en el historial.
- `git branch <nombre>` : Crea una rama.
- `git switch <rama>` : Cambia de rama.
- `git push / pull` : Sube o baja cambios del remoto.